



Anthropic API with Python - My Learning Notes

1. Why Python?

- Popular language for AI, backend development, automation, and rapid prototyping.
- Easy to understand and ideal for learning APIs.
- The same API concepts apply to other languages such as: Python, TypeScript, Java, Go, Ruby, C#, PHP, Command Line (CLI).

Key Takeaway: The programming language changes, but the API workflow remains the same.

2. Standard API Workflow

Every API request follows the same pattern:

1. Create an API client.
2. Select a model.
3. Send messages.
4. Configure parameters (e.g., `max_tokens`).
5. Receive and process the response.

3. Development Environment

For learning, the course uses: Visual Studio Code & Jupyter Notebook Extension.

- Benefits: Execute code one cell at a time, view outputs immediately, debug API requests easily, inspect responses interactively.

For production applications, you would typically use: FastAPI, Flask, Django, Regular Python scripts.

4. API Key Security Best Practices

Never expose your API key by:

- ✗ Hardcoding it in Python files
- ✗ Storing it in frontend JavaScript
- ✗ Committing it to GitHub

Instead:

- ✓ Store it in a `.env` file
- ✓ Load it using `python-dotenv`
- ✓ Add `.env` to `.gitignore`

Production environments should use: Environment Variables, Secrets Manager, Vault Services.

Golden Rule: Keep secrets outside your source code.

5. Sample Project Structure

```
ShopAssistAI/  
├─ lesson01.ipynb  
├─ .env  
├─ .gitignore  
└─ requirements.txt
```

6. Required Python Packages

```
pip install anthropic python-dotenv
```

- *anthropic*: Official Anthropic Python SDK.

- *python-dotenv*: Loads environment variables from *.env*.

7. Loading Environment Variables

```
from dotenv import load_dotenv
load_dotenv()
```

This loads the API key automatically from the .env file.

8. Creating the Anthropic Client

```
from anthropic import Anthropic
client = Anthropic()
```

The SDK automatically looks for: `ANTHROPIC_API_KEY`. No need to manually pass the API key in your code.

9. Selecting the Model

```
model = "claude-sonnet-4-6"
```

Best Practice: Store the model name in a variable so it can be changed easily later.

10. Sending Your First Message

```

message = client.messages.create(
    model=model,
    max_tokens=300,
    messages=[
        {
            "role": "user",
            "content": "A customer wants to return an order."
        }
    ]
)

```

This is the first real interaction between Python and Claude.

11. Important API Parameters

1. *model*

Specifies which Claude model to use. (Example: `model="claude-sonnet-4-6"`)

2. *max_tokens*

Limits the maximum number of generated tokens. (Example: `max_tokens=300`)

- *Important: It is NOT a target. Claude stops only if it reaches this limit.*

3. *messages*

Represents the conversation history. Each message includes: Role (user, assistant) and Content.

12. Printing Claude's Response

```
print(message.content[0].text)
```

Expected output: Hi there, I'm sorry to hear you'd like to return your order...

13. Understanding Stop Reasons

Every response includes a stop reason: `print(message.stop_reason)`

- `end_turn`: Claude completed its response naturally. (Safe to display)
- `max_tokens`: Claude stopped because it reached the token limit. (Response may be incomplete)
- `tool_use`: Claude is requesting the application to execute a tool (e.g., Search order DB, process refund). The application executes the tool before Claude continues.

14. Inspecting the Entire Response

`print(message)` - Useful during development because it reveals:

- Response ID, Model name, Generated content, Assistant role
- Stop reason, Token usage, Metadata

Example Output of `print(message)`:

```
Message(
  id='msg_01XyZ123AbC456',
  content=[TextBlock(text="Hi there,\n\nI'm sorry to hear you'd
like to return your order...", type='text')],
  model='claude-sonnet-4-6',
  role='assistant',
  stop_reason='end_turn',
  stop_sequence=None,
  type='message',
  usage=Usage(input_tokens=18, output_tokens=25)
)
```

15. Course Project: ShopAssist AI

Throughout the course, the API will power an AI assistant capable of: Order tracking, Returns, Refunds, Product recommendations, Customer support, and Order lookups using tools.

16. Key Learning Summary & Takeaways

- ✓ *API keys should never be hardcoded. Keep configuration separate.*
- ✓ *Every Claude API interaction follows a consistent request-response pattern.*
- ✓ *messages represents structured conversation history.*
- ✓ *max_tokens sets an upper limit, not a target.*
- ✓ *Inspecting the full response object is invaluable during development.*
- ✓ *Understanding stop_reason helps applications decide the next action.*